

# Programozás Alapjai

Dr. Gergely Tamás  
Dr. Kicsi András

Szegedi Tudományegyetem  
Informatikai Intézet  
Szoftverfejlesztés Tanszék

2025

(v1207)



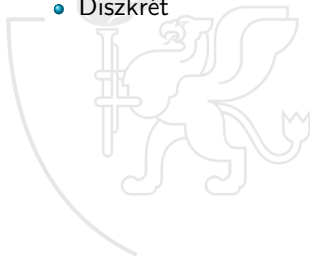
# Tartalomjegyzék

- 
- 1 Bemutatók**
    - Kurzus információk
    - Infrastruktúra szabályzatok
    - Mentorálás
  - 2 Python alapok**
    - Bevezető
    - Pénzváltás (1. verzió)
    - Pénzváltás (2. verzió)
  - 3 Alapfogalmak**
    - Alapfogalmak
    - Alapvető programelemek fogalmak
  - 4 Python feltételes ciklusok**
    - Röppálya számítás
    - Röppálya szimuláció
  - 5 Szoftverfejlesztési életciklus modellek**
    - A programozás fázisai
  - 6 Python sztringek**
    - Python sztringek
    - Basename meghatározása
  - 7 Vezérlési szerkezetek**
    - Bevezetés
    - Szekvenciális vezérlés
    - Függvények
  - 8 Python listák**
    - Python list és tuple adattípusok
    - Python listaszerű adattípusok műveletei
    - Basename és eltelt idő
  - 9 Vezérlési szerkezetek**
    - Szelekciós vezérlések
  - 10 Python iterációk**
    - Csúszóátlag számítása
  - 11 Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 12 Python halmazok, comprehension**
    - Python set és frozenset adattípusok
    - Tér legtávolabbi pontjai
  - 13 Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 14 Python dictionary, függvény mint típus**
    - Python dict adattípus
    - Python comprehension
    - Python function adattípus
  - 15 Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 16 Python fájlkezelés**
    - Fájlkezelés
  - 17 Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 18 Python formátált IO**
    - Python formátált IO
  - 19 Adatkezelés, adattárolás**
    - Az adatkezelés szintjei
    - Az adattárolás jellemzői
  - 20 Python típusok**
    - Python numerikus típusok
  - 21 Absztrakt adattípusok**
    - Egyszerű adattípusok
    - Összetett adattípusok
  - 22 Python típusok**
  - 23 Absztrakt adattípusok**
    - További Python típusok
    - Python function adattípus
    - Összetett adattípusok
  - 24 Tesztelés**
    - Tesztelés
    - Tesztelés Python-ban
    - A pytest
  - 25 Vizualizáció és egyéb modulok**
    - Fájlkezelő modulok
    - Vizualizáció
  - 26 More Python**
    - A Python program végrehajtásának folyamata
    - Függvényparaméterek és argumentumok
    - Python unpacking
    - Kivételkezelés

# Tartalomjegyzék

- 
- 1 **Bemutakozás**
    - Kurzus információk
    - Infrastruktúra szabályzatok
    - Mentorálás
  - 2 **Python alapok**
    - Bevezető
    - Pénzváltás (1. verzió)
    - Pénzváltás (2. verzió)
  - 3 **Alapfogalmak**
    - Alapfogalmak
    - Alapvető programelemek fogalmak
  - 4 **Python feltételes ciklusok**
    - Röppálya számítás
    - Röppálya szimuláció
  - 5 **Szoftverfejlesztési életciklus modellek**
    - A programozás fázisai
  - 6 **Python sztringek**
    - Python sztringek
    - Basename meghatározása
  - 7 **Vezérlési szerkezetek**
    - Bevezetés
    - Szekvenciális vezérlés
    - Függvények
  - 8 **Python listák**
    - Python list és tuple adattípusok
    - Python listaszerű adattípusok műveletei
    - Basename és eltelt idő
  - 9 **Vezérlési szerkezetek**
    - Szelekciós vezérlések
  - 10 **Python iterációk**
    - Csúszóátlag számítása
  - 11 **Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 12 **Python halmazok, comprehension**
    - Python set és frozenset adattípusok
    - Tér legtávolabbi pontjai
  - 13 **Vezérlési szerkezetek**
    - Ismétléses vezérlések
  - 14 **Python dictionary, függvény mint típus**
    - Python dict adattípus
    - Python comprehension
    - Python function adattípus
  - 15 **Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 16 **Python fájlkezelés**
    - Fájlkezelés
  - 17 **Vezérlési szerkezetek**
    - Eljárásvezérlés
  - 18 **Python formátált IO**
    - Python formátált IO
  - 19 **Adatkezelés, adattárolás**
    - Az adatkezelés szintjei
    - Az adattárolás jellemzői
  - 20 **Python típusok**
    - Python numerikus típusok
  - 21 **Absztrakt adattípusok**
    - Egyszerű adattípusok
    - Összetett adattípusok
  - 22 **Python típusok**
  - 23 **Absztrakt adattípusok**
    - Összetett adattípusok
  - 24 **Tesztelés**
    - Tesztelés
    - Tesztelés Python-ban
    - A pytest
  - 25 **Vizualizáció és egyéb modulok**
    - Fájlkezelő modulok
    - Vizualizáció
  - 26 **More Python**
    - A Python program végrehajtásának folyamata
    - Függvényparaméterek és argumentumok
    - Python unpacking
    - Kivételkezelés
  - További Python típusok
  - Python function adattípus

- Ismétléses vezérlésen olyan vezérlési előírást értünk, amely adott műveletnek adott feltétel szerinti ismételt végrehajtását írja elő.
- Az ismétlési feltétel szerint ötféle ismétléses vezérlést különböztetünk meg:
  - Kezdőfeltételes
  - Végfeltételes
  - Számlálásos
  - Hurok
  - Diszkrét



- Az algoritmustervezés során a leginkább megfelelő ismétléses vezérlési formát használjuk, függetlenül attól, hogy a megvalósításra használt programozási nyelvben közvetlenül megvalósítható-e ez a vezérlési mód.
- Ismétléses vezérlés képzését ciklusszervezésnek is nevezik, így az ismétlésben szereplő műveletet ciklusmagnak hívjuk.



- Problémafelvetés

- A globális felmelegedés a nyakunkon van, ez egyre inkább érezhető. Vannak viszont, akiknek ez az „érzés” nem elegendő, csak a bizonyítékoknak hisznek. (És vannak, akik azoknak sem, de az már messze vezet.) A meteorológusok a több, mint száz éve rendszeresen naponta többször mért adatokból szeretnének statisztikákat összeállítani, amik a napi, heti, havi, negyedéves minimum, maximum és átlaghőmérsékleteket mutatják, hogy ezek változásával mutassák ki a jellemző trendeket. Egy-egy ilyen adathármas előállításához sok-sok (adott intervallumba eső) hőmérsékleti értéket kell feldolgozni, ezért ehhez számítógépes segítséget kérnek.
- Kellene egy program, amely meghatározza egy valós számsorozat legkisebb és legnagyobb elemét, valamint a sorozat átlagát!

- Specifikáció

- A probléma inputja egy valós számokból álló sorozat.
- Az input számsorozat végét egy üres input sor fogja jelezni.
- Az output a sorozat legkisebb és legnagyobb eleme, valamint az átlaga.



# Számsorozat jellemzői

## Algoritmustervezés 1.

- Elsőre talán az tűnne a legegyszerűbb megoldásnak, ha beolvasnánk az összes számot, majd ezek között megkeresnénk a legkisebbet, legnagyobbat, majd kiszámolnánk az átlagot.
- Sőt, ha a számsorozatot eleve parancssori argumentumként kérjük, és kihasználjuk a Python nyelv lehetőségeit, akkor elég könnyű dolgunk van.



# Számsorozat jellemzői

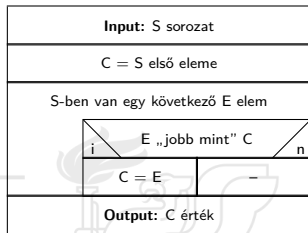
minimax\_args.py [1–24]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Számsorozat minimumának, maximumának és átlagának meghatározása
9"""
10import sys
11
12def main():
13    print("Ez a program valós számsorozat minimális,")
14    print("maximális elemét és átlagát számolja.")
15
16    sorozat = list(map(float, sys.argv[1:]))
17    if len(sorozat) == 0:
18        print("Üres számsorozat érkezett.")
19    else:
20        print(f"Minimum = {min(sorozat):10.3f} Maximum = {max(sorozat):10.3f}")
21        print(f"Az átlag = {sum(sorozat)/len(sorozat):10.3f}")
22
23if __name__ == "__main__":
24    main()
```

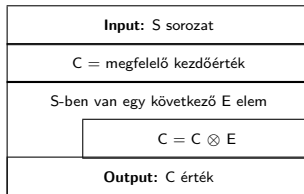
- De gondoljuk végig, hogyan is működne a három részalgoritmus?
  - Minimum keresése:
    - Legyen kezdetben a *minimum* az első elem.
    - Vizsgáljuk sorban a sorozat elemeit. Ha a vizsgált elem kisebb a *minimum*-nál, akkor ez lesz az új *minimum*. Az összes elem feldolgozása után a *minimum* a teljes sorozat minimuma lesz.
  - A maximális elem keresése ehhez teljesen hasonlóan megy.
  - Az átlagszámításhoz össze kell adni a sorozat összes elemét, majd a végén el kell osztani a sorozat elemeinek számával.
    - Legyen az *összeg* és az *elemszám* kezdetben nulla.
    - Vizsgáljuk sorban a sorozat elemeit. Minden elemnél az elem értékét adjuk hozzá az *összeghez*, és növeljük eggyel az *elemszámot*.

- Ha végiggondoljuk, a három részalgoritmus összevonható.
- A soron következő elem feldolgozásával ki tudjuk számolni az eddigi sorozat minimumát, maximumát, összegét és elemszámát.
- Ha ezt megtettük, a sorozat ezen elemére nincs többé szükség, azt nem kell eltárolni.
  - Nem lesz szükségünk az elemek listájára, kevesebb memóriát használ a programunk.
  - Olyan sorozatokat is fel tudunk dolgozni, amelyek nem férnének el a memóriában.
- Megjegyeznénk, hogy egy sorozat minimumának/maximumának kiválasztása, illetve elemeiből egyetlen érték előállítása úgynevezett *programozási tételek*, vagyis nagyon alapvető algoritmusok. Itt konkrétan a *szélsőérték-kiválasztás* (minimum és maximum keresés), az *összegzés* (elemek összegének előállítása) és a *számlálás* (a sorozat darabszáma) algoritmusokat használjuk/kombináljuk.

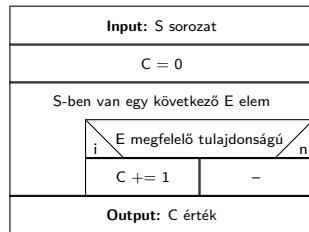
- Szélsőérték-keresés



- Összegzés



- Számlálás



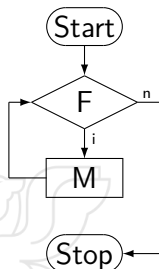
# Kezdőfeltételes ismétléses vezérlés

- Kezdőfeltételes vezérlésről akkor beszélünk, ha a ciklusmag (ismételt) végrehajtását egy belépési (ismétlési) feltételhez kötjük.
- Legyen  $F$  logikai kifejezés,  $M$  pedig tetszőleges művelet. Az  $F$  ismétlési feltételből és az  $M$  műveletből (a ciklusmagból) képzett kezdőfeltételes ismétléses vezérlés a következő vezérlési előírást jelenti:
  1. Értékeljük ki az  $F$  feltételt.
  2. Ha  $F$  értéke hamis, akkor az ismétlés és ezzel együtt az összetett művelet végrehajtása befejeződött.
  3. Egyébként, vagyis ha az  $F$  értéke igaz, akkor hajtsuk végre az  $M$  műveletet, majd folytassuk az 1.) lépéssel.

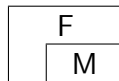
# Kezdőfeltételes ismétléses vezérlés

## Folyamatábra és struktogram

- Folyamatábrája:



- Struktogramja:

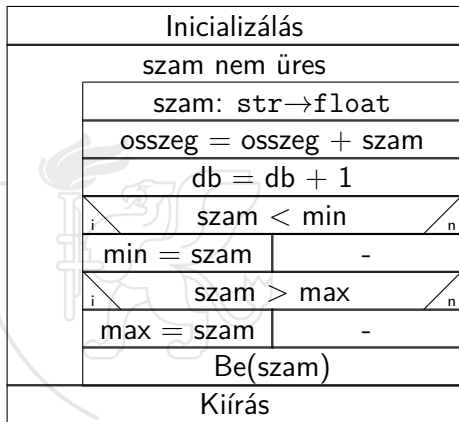


- Ha az  $F$  értéke kezdetben hamis,  $M$  egyszer sem kerül végrehajtásra.
- Ha az  $F$  értéke kezdetben igaz, és az  $M$  művelet nincs hatással  $F$ -re, akkor végtelen ciklust kapunk.

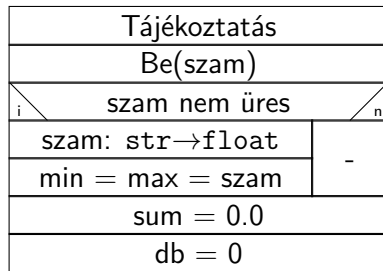
# Számsorozat jellemzői

## Algoritmustervezés – Struktogram

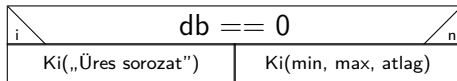
- A főprogram:



- Inicializálás:



- Kiírás:



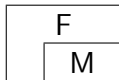
# A while utasítás

- Ha valamilyen műveletet mindaddig végre kell hajtani, amíg egy feltétel igaz a **while** utasítás használható.
  - A művelet végrehajtása nem szükséges a feltétel kiértékeléséhez.
  - A feltétel ellenőrzése a művelet előtt történik, így ha a feltétel kezdetben hamis volt, a műveletet egyszer sem hajtjuk végre.
- **while utasítás:** A **while** utasítás Python-ban egy **while blokk**-ból és utána nulla vagy egy **else blokk**-ból állhat  
(`[[[:while blokk:]] [[[:else blokk:]]?]`).
- **while blokk:** A 'while' kulcsszó, utána egy kifejezés és egy ':', majd a következő sortól egy indentált kódblokk  
(`'while' [[[:Kifejezés:]] ':' '\n' [[[:Indentált kódblokk:]]]`).
- **else blokk:** Az 'else' kulcsszó és egy ':' majd a következő sortól egy indentált kódblokk  
(`'else' ':' '\n' [[[:Indentált kódblokk:]]]`).

# Kezdőfeltételes ismétléses vezérlés

Megvalósítás Python nyelven

- Struktogramja:



- Megvalósítása:

```
while F:  
    M
```



# A Python értékadó utasítása

- A Python nyelv egyik jó tulajdonsága, hogy a szintaxisa kevésbé rigid, és néhol tömör kód írását is lehetővé teszi (ami persze néha szembemegy a „Sparse is better than dense.” filozófiával, de kit érdekel).
- Mielőtt elhamarkodottan lekódoznánk az előző struktogramon található programot, ismerkedjünk meg a Python nyelv néhány ilyen konstrukciójával.



# A Python értékadó utasítások

- Python-ban a

```
v = v ⊙ e
```

alakú értékadásokat sok esetben

```
v ⊙ = e
```

alakban rövidíthetjük, ahol a  $\odot$  tetszőleges műveletet jelölhet.

```
>>> x = 5
```

```
>>> x
```

```
5
```

```
>>> x += 2
```

```
>>> x
```

```
7
```

```
>>> x *= 6
```

```
>>> x
```

```
42
```

```
>>> x -= 10
```

```
>>> x
```

```
32
```

```
>>> x **= 2
```

```
>>> x
```

```
1024
```

```
>>> x /= 100
```

```
>>> x
```

```
10.24
```

```
>>> x %= 4
```

```
>>> x
```

```
2.24
```

```
>>> x //= 0.5
```

```
>>> x
```

```
4.0
```

# A Python értékadó utasítások

- A  $v = v \odot e$  és  $v \odot = e$  műveletek egyenértékűsége Python-ban típusfüggő.
- Változó értékének cseréje (*immutable* típusoknál csak ez lehetséges)
- Változó értékének módosítása (csak *mutable* típusoknál lehetséges)

```
>>> a = b = (1, 2)
>>> x = y = (1, 2)
>>> a is b, x is y
(True, True)
>>> a += (3, 4)
>>> a, b
((1, 2, 3, 4), (1, 2))
>>> x = x + (1, 2)
>>> x, y
((1, 2, 3, 4), (1, 2))
>>> a is b, x is y
(False, False)
```

```
>>> a = b = [1, 2]
>>> x = y = [1, 2]
>>> a is b, x is y
(True, True)
>>> a += [3, 4]
>>> a, b
([1, 2, 3, 4], [1, 2, 3, 4])
>>> x = x + [3, 4]
>>> x, y
([1, 2, 3, 4], [1, 2])
>>> a is b, x is y
(True, False)
```

# Értékadó utasítások és műveletek

- Az egyszerű értékadó utasítás halmazható, több változónak adhatjuk ugyanazt az értéket.
- Az értékadás (=) ettől még utasítás, nem művelet!
- Az értékadó művelet a :=, „rozmár” operátor.

```
a = b = c = 6 * 7  
a = b = (c = 6 * 7) # EZ HIBÁS!!!  
a = (b := (c := 6) * 7)
```

- A műveletekkel kombinált értékadó utasítások nem halmazhatók, csak önálló utasításként használhatók.

```
a = 6  
b = 7  
a *= b  
a += b += 3 # EZ HIBÁS!!!
```

# Számsorozat jellemzői [1/2]

minimax.py [1–19]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Számsorozat minimumának, maximumának és átlagának meghatározása
9"""
10def main():
11    print("Ez a program valós számsorozat minimális,")
12    print("maximális elemét és átlagát számolja.")
13    print("Az input sorozatot egy üres sor zárja.")
14    print("Kérem az input számsorozatot!")
15    szam = input("? ")
16    if szam:
17        s_min = s_max = float(szam)
18    s_sum = 0.0
19    db = 0
```

# Számsorozat jellemzői [2/2]

minimax.py [21–40]

```
21 while szam:
22     szam = float(szam)
23     s_sum += szam
24     db += 1
25
26     if szam < s_min:
27         s_min = szam
28     elif szam > s_max:
29         s_max = szam
30
31     szam = input("? ")
32
33 if db == 0:
34     print("Üres számsorozat érkezett.")
35 else:
36     print(f"Minimum={s_min:10.3f} Maximum={s_max:10.3f}")
37     print(f"Az átlag={s_sum/db:10.3f}")
38
39 if __name__ == "__main__":
40     main()
```

# Szinusz(x) kiszámítása

## Problémafelvetés és specifikáció

- Problémafelvetés

- Egy processzorgyártó cég az elhíresült *Pentium FDIV bug* óta különös figyelmet szentel a processzorok működésének átfogó tesztelésére. Jelenleg éppen a trigonometrikus számításokat ellenőrzik. Ezt úgy próbálják megtenni, hogy a szinusz(x) értékét az alapvető műveletekkel kiszámoltatják (adott pontosságon), és ezt majd összehasonlítják a dedikáltan ilyen számításokat végző processzorműveletek eredményével.
- Ehhez kellene egy olyan program, ami képes kiszámolni a szinusz(x) közelítő értékét a 4 matematikai alpművelet segítségével, a processzorba épített trigonometrikus képességek használata nélkül.

- Specifikáció

- A probléma inputja egy  $x$  valós szám.
- Az output a  $\sin(x)$  közelítő értéke.
- Az eredményt  $10^{-10}$  pontossággal kell kiszámítani.
- A szinusz kiszámításához nem használhatjuk a beépített trigonometrikus függvényeket.

# Szinusz(x) kiszámítása

## Algoritmustervezés 1.

- Mint az köztudott,

$$\sin(x) = \sum_{i=0}^{\infty} -1^i \frac{x^{(2i+1)}}{(2i+1)!}$$

A  $\sin(x)$  értéke közelíthető ezen sor egy véges kezdőszeletével.

- Akármeekkora legyen is a sorfejtés következő tagja, az  $\frac{x-1}{2}$ -ik tag után a további tagok értéke monoton csökkenő lesz. Mivel alternáló összegről van szó, ha a következő tag értéke kisebb az elvárt pontosságnál, biztosak lehetünk benne, hogy a  $\sin(x)$  értékét legalább ilyen pontossággal megközelítettük, tehát befejezhetjük a számolást.

# Szinusz(x) kiszámítása

## Algoritmustervezés 2.

- Nyilvánvalóan nem célszerű az összes tagot egyedileg kiszámolni, hiszen a következő tag viszonylag egyszerűen számolható az előzőből, és még időt is spórolunk az előző érték felhasználásával.
- Ráadásul a tag számlálójának és nevezőjének külön számolása még pontatlanná is tenné a számítást, mert mindkettő gyorsan növekszik  $i$  függvényében.
- A float típus pontatlansága miatt még így is figyelni kell arra, hogy ne legyen túl nagy eltérés a számolás egyes részeredményei között. Ezt elérhetjük úgy, hogy kihasználva a szinusz periodikusságát, a lehető legkisebb abszolút értékű  $x$ -re számoljuk ki a  $\sin(x)$  értékét úgy, hogy az eredeti érték szinuszával megegyező eredményt kapjunk.
- Itt is megjegyeznénk, hogy az összegzés programozási tételét alkalmazzuk, több szinten is.

# Végfeltételes ismétléses vezérlés

- Végfeltételes vezérlésről akkor beszélünk, ha a ciklusmag elhagyását egy kilépési feltételhez kötjük.
- Legyen  $F$  logikai kifejezés,  $M$  pedig tetszőleges művelet. Az  $F$  kilépési feltételből és az  $M$  műveletből (a ciklusmagból) képzett végfeltételes ismétléses vezérlés a következő vezérlési előírást jelenti:
  - 1 Hajtsuk végre az  $M$  műveletet.
  - 2 Értékeljük ki az  $F$  feltételt.
  - 3 Ha  $F$  értéke igaz, akkor az ismétléses vezérlés és ezzel együtt az összetett művelet végrehajtása befejeződött.
  - 4 Egyébként, vagyis ha az  $F$  értéke hamis, akkor folytassuk az 1.) lépéssel.

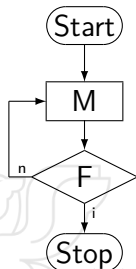
# A break és continue utasítás

- A Python nyelvben a ciklusmag folyamatos végrehajtásának megszakítására két utasítás használható:
  - break** Megszakítja a ciklust, a program végrehajtása a ciklus utáni első utasítással folytatódik. A ciklushoz tartozó else ág **nem** hajtódik végre.
  - continue** Megszakítja a ciklusmag aktuális lefutását, a vezérlés a ciklus feltételének kiértékelésével (`while`) illetve a soron következő elem feldolgozásával (`for`) folytatódik.
- **break utasítás:** A 'break' kulcsszó ('break').
- **continue utasítás:** A 'continue' kulcsszó ('continue').

# Végfeltételes ismétléses vezérlés

## Folyamatábra és struktogram

- Folyamatábrája:



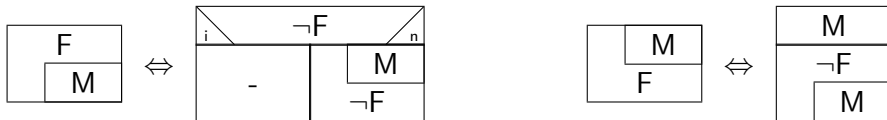
- Struktogramja:



- Ha az  $F$  értéke kezdetben hamis, és az  $M$  művelet nincs hatással  $F$ -re, akkor végtelen ciklust kapunk.
- Ha az  $F$  értéke kezdetben igaz,  $M$  legalább egyszer akkor is végrehajtásra kerül.

# Kezdő- és végfeltételes vezérlések kapcsolata

- A kezdő és végfeltételes vezérlések kifejezhetők egymás segítségével.

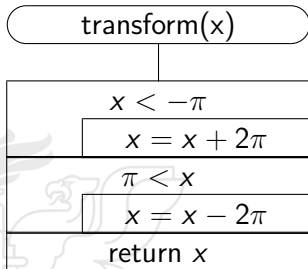


- Az algoritmus tervezésekor előfordulhat, hogy a kezdő- és végfeltételes vezérlés is alkalmasnak látszik a probléma megoldására. Ilyenkor érdemes megvizsgálni, hogy az  $F$  feltétel szükséges feltétele-e az  $M$  művelet végrehajtásának? Ha igen, akkor valószínűleg kezdőfeltételes ismétléses vezérlést kell választani.

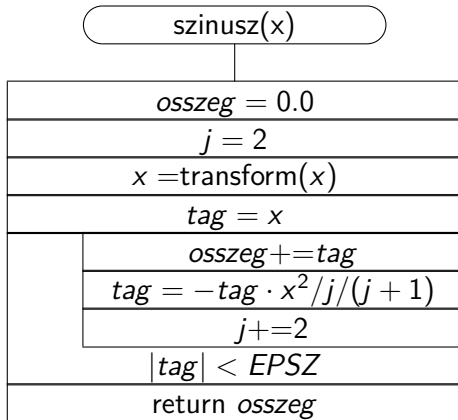
# Szinusz(x) kiszámítása

## Algoritmustervezés – Strugtogram

- A transform függvény:



- A szinusz függvény:



# Végfeltételes ismétléses vezérlés

Megvalósítás Python nyelven

- Struktogramja:



- Megvalósítása:

```
while True:
    M
    if F:
        break
```



# Szinusz [1/2]

szinusz.py [1-20]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-06-04
6cc-by-nc-nd
7
8sin(x) közelítő értékének kiszámítása a sin(x) függvény alkalmazása nélkül.
9x értékét transzformáljuk a [-Pi, Pi] intervallumba.
10"""
11from math import pi
12
13EPSZ = 1e-10
14
15def transform(x):
16    while x < -pi:
17        x += 2.0 * pi
18    while pi < x:
19        x -= 2.0 * pi
20    return x
```

# Szinusz [2/2]

szinusz.py [22-45]

```
22 def szinusz(x):
23     global EPSZ
24     osszeg = 0.0
25     j = 2
26
27     x = transform(x)
28
29     tag = x
30     xx = x ** 2
31
32     while True:
33         osszeg += tag
34         tag = -(tag * xx / j / (j + 1))
35         j += 2
36         if abs(tag) < EPSZ:
37             break
38     return osszeg
39
40 def main():
41     x = float(input("Kérem sin(x)-hez az x értéket: "))
42     print("sin(%.5f)= %.10f\n" % (x, szinusz(x)))
43
44 if __name__ == "__main__":
45     main()
```